

IBM INTERNSHIP PROJECT

Agentic AI Helpdesk

An Agentic AI-Powered IT Support and Helpdesk Assistant using RAG and ReAct Tool Orchestration

Project Domain: Agentic AI and IT Support

Framework: RAG and ReAct Tool Orchestration

Implementation Language: Python 3.14

Prepared according to the IBM Internship Project Template

1. PROJECT OVERVIEW

Project Title

Agentic AI Helpdesk: An Agentic AI-Powered IT Support and Helpdesk Assistant using RAG and ReAct Tool Orchestration

Problem Statement

IT support teams frequently handle repetitive technical issues such as VPN disconnections, Wi-Fi connectivity problems, account lockouts, system crashes, printer issues, and RAM or CPU performance complaints. Traditional helpdesk workflows often depend on manual troubleshooting, knowledge-base searches, and technician intervention, which can increase resolution effort and delay assistance.

Users need a support system that can understand an IT issue, identify relevant troubleshooting procedures, perform appropriate diagnostics, automate routine support actions, and escalate unresolved or urgent incidents.

Brief Description of the Project

Agentic AI Helpdesk is an AI-powered IT support system that combines Retrieval-Augmented Generation (RAG), agentic tool orchestration, and Large Language Models. A user can submit an IT issue through Web Chat or CLI, after which the system retrieves relevant Standard Operating Procedure (SOP) articles from its knowledge base.

The agent can autonomously select diagnostic and support actions according to the issue type. Network and VPN issues can trigger ping and DNS diagnostics, account lockouts can trigger a password-reset workflow, and unresolved or urgent issues can result in automatic IT ticket creation. The final response combines retrieved knowledge, tool outputs, and AI-generated guidance into a structured resolution report.

2. OBJECTIVES & PROPOSED SOLUTION

Project Objectives

- **To automate assistance:** for common IT support issues and repetitive troubleshooting requests.
- **To retrieve relevant SOPs:** from the knowledge base using TF-IDF vectorization and cosine similarity.
- **To automate diagnostics:** including gateway ping, DNS lookup, disk storage, and memory checks.
- **To automate support actions:** such as password-reset token generation and IT incident creation.
- **To manage tickets:** with identifiers, priorities, categories, statuses, and resolution notes.
- **To provide transparency:** through a live execution trace showing agent actions and tool outputs.
- **To support flexible AI:** through OpenAI, Google Gemini, or the built-in Smart Offline Fallback Engine.

Proposed Solution

The proposed solution is an Agentic AI Helpdesk system that combines a retrieval layer, an AI decision layer, and deterministic support tools. Instead of returning only a conversational answer, the agent can retrieve evidence, select a relevant tool, inspect its output, and then synthesize a resolution.

The workflow uses a JSON-based knowledge base containing SOP articles and a persistent JSON ticket database. The RAG component converts the incoming query and knowledge articles into TF-IDF vectors and ranks them using cosine similarity. The agent then uses ReAct-style tool orchestration to determine whether diagnostics, password reset, or ticket creation is required.

How the Agentic AI Solution Works

Step 1 – Query Ingestion: The user submits an IT issue through Web Chat or CLI.

Step 2 – RAG Knowledge Search: The query is converted to TF-IDF vectors and matching SOP articles are retrieved using cosine similarity.

Step 3 – Issue Analysis: The agent determines the issue category and identifies the appropriate support action.

Step 4 – Tool Orchestration: The agent invokes diagnostics, password reset, or IT ticket tools according to the situation.

Step 5 – Diagnostic Execution: Network issues can trigger ping and DNS tests, while system checks can inspect storage and memory.

Step 6 – Ticket Escalation: Unresolved or urgent issues can automatically create a prioritized incident ticket.

Step 7 – Solution Synthesis: The LLM combines retrieved SOPs and tool outputs into a structured resolution report.

Step 8 – Knowledge Update: New SOP articles can be published and the RAG index can be rebuilt dynamically.

3. NOVELTY AND KEY FEATURES

Novelty: Agentic RAG with Autonomous IT Tool Orchestration

The main novelty of the Agentic AI Helpdesk is the combination of RAG-based troubleshooting knowledge with an agentic tool-execution workflow. Instead of treating every request as a simple question-answering task, the system can retrieve relevant SOP information and select an appropriate operational action based on the issue.

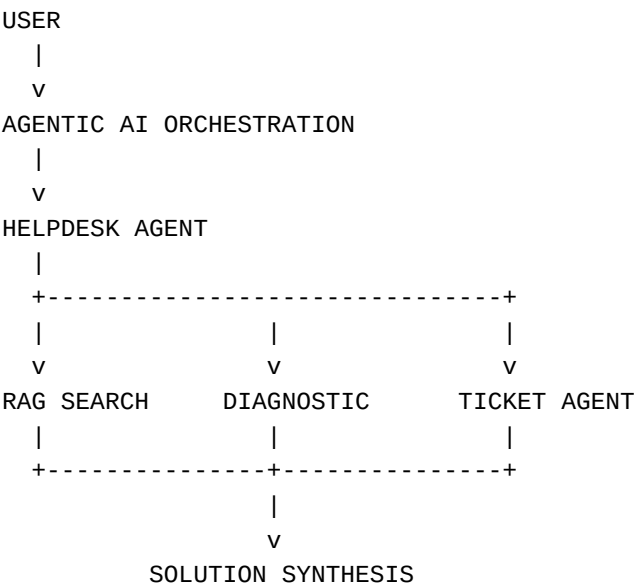
The agent can connect the retrieved context to deterministic tools such as network diagnostics, password-reset automation, and incident-ticket creation. This creates a workflow in which the AI can gather evidence, perform an action, and use the result when generating the final support response.

This approach creates a feedback-driven support process rather than a simple chatbot or fixed knowledge-base search.

Key Features

- **Semantic RAG Knowledge Search:** Retrieves troubleshooting SOPs for VPN, BSOD, Wi-Fi, Outlook, Account Lockouts, RAM/CPU spikes, Printer Spooler, and UAC administrator rights.
- **Automated Diagnostic Suite:** Provides gateway ping, DNS lookup, disk storage, and memory usage checks.
- **Ticket Lifecycle Management:** Creates INC-xxxx tickets with priority, category, status, and resolution-note information.
- **Self-Service Password Reset:** Generates account verification tokens and reset links for account lockout scenarios.
- **Live Execution Trace:** Displays step-by-step agent actions and tool execution outputs.
- **Dynamic KB Publishing:** Allows new SOP articles to be published and the RAG index to be rebuilt on the fly.

Proposed Agent Architecture



|
v

STRUCTURED RESOLUTION

4. IMPLEMENTATION & RESULTS

Technologies / Tools Used

Technology / Tool	Purpose
Python 3.14	Core programming language
HTML5 / Bootstrap 5.3 / JavaScript	Responsive web user interface
web_server.py	Native REST API server
Streamlit (app.py)	Alternative interactive web interface
scikit-learn	TF-IDF vectorization and cosine similarity retrieval
OpenAI / Google Gemini	LLM-based reasoning and response generation
Offline Fallback Engine	AI support when an external LLM is unavailable
Python unittest	Automated testing and verification
JSON	Knowledge base and persistent ticket storage

Working Process

- **Query Ingestion:** The user submits an IT issue through Web Chat or CLI.
- **RAG Knowledge Search:** The query is converted into TF-IDF vectors and matching SOP articles are retrieved.
- **Agent Decision:** The agent identifies the issue type and determines the next support action.
- **Diagnostic Execution:** Network/VPN issues can trigger ping and DNS checks; system diagnostics include storage and memory checks.
- **Password Reset:** Account lockout requests can invoke trigger_password_reset and generate a reset token.
- **Ticket Creation:** Unresolved or urgent issues can trigger create_it_ticket with an appropriate priority.
- **Solution Synthesis:** Retrieved knowledge and tool outputs are combined into a structured resolution.
- **Dynamic KB Publishing:** New SOP content can be added and the RAG vector index rebuilt.

Expected Screenshots / Output

The final implementation can include the following screenshots in the completed project:

1. Home page of the Agentic AI Helpdesk.
2. IT issue submission and chat interface.
3. RAG knowledge retrieval and relevance result.
4. Live agent execution trace and diagnostic output.
5. Generated incident ticket and ticket status.
6. Password-reset token generation output.

Results Achieved / Expected Demonstration

The implemented system was verified using the Python unittest suite, with **5/5 unit tests passed**, representing a **100% pass rate** in a reported execution time of **0.008 seconds**.

- **VPN Disconnections:** Successfully matched KB001 with 100% confidence and executed network ping/DNS diagnostics.
- **Account Lockouts:** Successfully invoked trigger_password_reset and generated a reset token in the RST-xxxx format.
- **Windows BSOD Crash:** Diagnosed stop code CRITICAL_PROCESS_DIED and automatically generated high-priority ticket INC-1004.
- **Ticket Query:** Successfully retrieved ticket INC-1001 with status “In Progress” and technician notes.

These scenarios demonstrate the core capabilities of the prototype: knowledge retrieval, issue-specific tool selection, automated diagnostics, password-reset workflow execution, ticket generation, and retrieval of existing ticket information. The results provide functional verification of the implemented helpdesk workflow.

5. CONCLUSION & FUTURE SCOPE

Project Conclusion

Agentic AI Helpdesk demonstrates a practical Agentic AI approach to IT support by combining RAG-based knowledge retrieval, LLM reasoning, and deterministic operational tools. Rather than functioning only as a conversational chatbot, the system can retrieve relevant SOPs, perform diagnostics, initiate password-reset actions, and create incident tickets when required.

The project provides multiple interfaces through a native REST API web application, Streamlit, and CLI. It also maintains a JSON-based knowledge base and persistent ticket database. The implemented test suite achieved a 100% pass rate, and practical scenarios verified VPN troubleshooting, account lockout handling, BSOD escalation, and ticket-status retrieval.

The project demonstrates how an agentic workflow can connect AI-generated reasoning with actionable IT support operations.

Challenges Faced

- Designing reliable retrieval of relevant SOP content for different IT queries.
- Coordinating LLM decisions with deterministic diagnostic and support tools.
- Maintaining persistent ticket and knowledge-base information using JSON storage.
- Supporting multiple LLM providers through a common flexible client.
- Providing an offline fallback when external LLM access is unavailable.
- Making agent actions and tool outputs transparent through execution traces.

Future Enhancements

- Advanced Embedding Retrieval: Extend TF-IDF retrieval with embedding-based vector search.
- Enterprise Database Integration: Replace JSON storage with a production database.
- Real System Diagnostics: Extend simulated checks to authorized real-world diagnostics.
- ITSM Integration: Connect the ticket workflow with enterprise IT service-management platforms.
- Authentication and Role-Based Access: Add employee, technician, and administrator roles.
- Advanced Agent Planning: Add specialized tools and multi-step troubleshooting workflows.
- Analytics Dashboard: Visualize incident volume, common issues, priorities, and resolution metrics.

References

1. Python Documentation – Python programming language and standard library.
2. scikit-learn Documentation – TF-IDF vectorization and cosine similarity.
3. OpenAI API Documentation – Large Language Model integration.
4. Google Gemini Documentation – Generative AI model integration.
5. Bootstrap Documentation – Responsive web interface development.
6. Streamlit Documentation – Python-based application interface development.
7. Python unittest Documentation – Automated software testing.